

# XQDot4Z: productos punto cuantizados en un procesador RISC-V

Autoría por definir

Borrador de trabajo v0.9 — no destinado a envío

**Resumen**—El almacenamiento de pesos con pocos bits reduce el tamaño de los datos, pero su procesamiento puede requerir extracción, conversión y corrección del origen numérico. Este trabajo plantea estudiar una operación que combine cuatro productos entre pesos de cuatro bits sin signo y activaciones de ocho bits con signo, con corrección del punto cero. La pregunta es cuándo esa fusión resulta útil frente a multiplicación escalar y frente a un producto punto empacado con corrección separada. Se propone utilizar un procesador docente de treinta y dos bits, verificar primero la operación y evaluar después productos punto y multiplicaciones matriz–vector. Este documento es un borrador inicial: presenta la pregunta y el método previsto, pero no reporta resultados de aceleración. Las conclusiones se redactarán cuando existan implementaciones verificadas y comparaciones reproducibles.

**Palabras clave**—inferencia cuantizada, aritmética de baja precisión, extensiones de instrucciones, verificación de hardware

## I. INTRODUCCIÓN

Las capas de una red neuronal realizan productos y sumas entre pesos y activaciones. La cuantización permite representar parte de ese cálculo con enteros y parámetros de escala [1]. Sin embargo, almacenar más pesos por palabra no elimina el trabajo necesario para consumirlos en un procesador. La extracción de campos y la preparación de los operandos pueden ser relevantes.

Se propone estudiar XQDot4Z sobre Kuntur, un procesador docente RISC-V con un pipeline de cinco etapas. La intención es investigar una operación acotada y sus costos, no desarrollar todavía un acelerador completo ni ejecutar un modelo de lenguaje de gran tamaño.

## II. FUNDAMENTOS Y TRABAJO RELACIONADO

En cuantización afín, un código entero  $q$  representa aproximadamente  $r = s(q - z)$ , donde  $s > 0$  es la escala y  $z$  el punto cero o *zero-point*. Con activaciones cuyo punto cero es cero, el núcleo aritmético estudiado es

$$d = \sum_{i=0}^3 (q_{w,i} - z_w) q_{a,i}. \quad (1)$$

Los pesos se almacenan como U4 (cuatro bits sin signo) y las activaciones como S8 (ocho bits con signo). La salida propuesta es un subtotal INT32 (entero de 32 bits con signo); escalas, sesgo y acumulación de otros subtotales se mantienen fuera de esta operación.

Entre los antecedentes están XpulpNN y Dustin, con instrucciones de baja precisión y aritmética mixta en RISC-V [2], [3]. Por ello, no se considera inédito el uso de

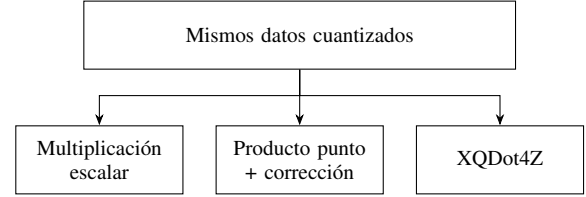


Fig. 1. Comparación propuesta sobre una base común. No representa resultados medidos.

productos punto de pocos bits. La contribución específica deberá establecerse mediante una comparación más detallada de antecedentes y los resultados propios.

Una alternativa importante a la fusión es

$$d = \sum_i q_{w,i} q_{a,i} - z_w \sum_i q_{a,i}. \quad (2)$$

La suma de activaciones puede reutilizarse entre filas de una matriz. Una comparación válida debe permitir esa optimización.

## III. PREGUNTA Y ALCANCE

*¿Bajo qué condiciones fusionar la corrección del punto cero en un producto punto U4 por S8 reduce su costo de ejecución frente a multiplicación escalar y corrección factorizada, considerando reutilización de activaciones y suministro del punto cero?*

El alcance inicial es un solo core, batch uno y kernels de producto punto y matriz–vector pequeños. La primera fase evaluará corrección y ciclos de simulación. La síntesis y la evaluación en una placa se consideran etapas posteriores. No se deducirá energía ni tiempo físico únicamente del número de instrucciones o ciclos.

## IV. MÉTODO PROPUESTO

### A. Plataforma y verificación

Se preserva el procesador original y se trabaja en un clon independiente. Las correcciones del core se verifican por separado y formarán una base común para las variantes. Las instrucciones no implementadas deberán detectarse, sin atribuir al procesador conformidad completa con una ISA que aún no cubre.

El contrato numérico v0.1 fija una palabra  $W$  de 32 bits con ocho pesos U4 y una palabra  $A$  con cuatro activaciones S8. Un selector  $h$  elige los cuatro pesos bajos o altos de  $W$ , sin cambiar las activaciones. El índice cero ocupa los bits menos

significativos y  $z_w$  es común a los cuatro términos. El subtotal está en  $[-7680, 7680]$  y se representa en complemento a dos de 32 bits, sin acumulación implícita.

Se dispone de una referencia Python con enteros sin ancho fijo, comprobada mediante ejemplos, pruebas de packing, ternas exhaustivas de un término y productos punto aleatorios contrastados con (2). La unidad RTL aislada es combinacional, con cuatro vías, productos S13 y un árbol de sumas con extensión explícita de signo, verificada en Icarus Verilog y Verilator. La interfaz experimental XQDot4Zi la integra opcionalmente en Kuntur, con dos registros fuente y  $z_w, h$  inmediatos. Reutiliza el forwarding y transporta los metadatos con la instrucción. No añade una espera propia en E, pero no se ha comprobado timing físico. El punto cero leído en ejecución sigue pendiente; un inmediato no lo sustituye gratuitamente.

La base incorpora MUL escalar opcional, independiente de XQDot4Zi, que devuelve los 32 bits bajos del producto [4]. No implementa M ni Zmmul completos. Es combinacional, reutiliza forwarding y se cancela con flush/reset; su disponibilidad funcional no demuestra timing físico.

El comparador empacado (B3) dispone de una unidad de cuatro vías sin resta del punto cero, con el mismo packing y selector que la variante fusionada XQDot4Z (D). Su subtotal está en  $[-7680, 7620]$ ; la corrección de (2) queda fuera de esa unidad. La instrucción opcional XQDot4 la integra con dos GPR fuente,  $h$  inmediato y el mismo forwarding. Con pesos iguales a uno, también permite obtener la suma de activaciones reutilizable entre filas.

### B. Comparación prevista

Se compararán multiplicación escalar, producto punto empacado con la corrección de (2) y XQDot4Z (Fig. 1). Las variantes recibirán los mismos datos y compartirán las correcciones del core. Se contará el trabajo de cargas, bucles, corrección y suministro de metadatos, además del núcleo aritmético.

El manifiesto inicial barre filas con un grupo de 32 elementos y separa controles constantes de códigos U4 equilibrados entre fases. Distingue cero, uno, otras potencias de dos y demás códigos; no atribuye tiempos de programas mixtos a esos estratos por frecuencia. Evalúa especialización estática, sin extrapolar a metadatos dinámicos. Desenrollado, registros, planificación y reutilización de correcciones deben fijarse antes de medir. Se reportarán todos los casos, incluidos fallos y degradaciones. Aún no hay kernels de rendimiento.

## V. RESULTADOS Y DISCUSIÓN

XQDot4Z aislada supera 556 734 comparaciones por simulador, ocho controles negativos y cuatro mutaciones en Icarus; el lint no requiere excepciones. La exhaustividad solo corresponde a ternas de un término. La integración inmediata supera 25 programas, incluidas 1056 operaciones numéricas, con comparación de retiros, writeback, stores y operandos, y dos mutaciones detectadas. MUL supera 86 704 vectores por simulador y 49 programas con cuatro configuraciones,

ocho controles negativos por simulador y cuatro mutaciones en Icarus. Se conservan artefactos, logs y hashes de estas baterías.

La unidad empacada sin corrección supera 67 910 vectores por simulador, con diez controles negativos por simulador y cuatro mutaciones en Icarus. A partir de 1680 respuestas RTL se reconstruyen 336 salidas por fila/grupo en 24 casos matriz–vector, reutilizando la suma de activaciones entre filas. En esa batería aislada, la acumulación y corrección se realizan en el verificador.

La integración de XQDot4 supera 123 programas en ambos simuladores, con las ocho configuraciones de MUL y las dos instrucciones empacadas, y detecta tres mutaciones en Icarus. Cuatro pares de fixtures de dos filas producen ocho salidas iguales en B3 y D. En B3, Kuntur calcula una sola suma de activaciones, la reutiliza y corrige mediante MUL/SUB; D acumula subtotales fusionados. Son pruebas de exactitud con punto cero conocido al generar el código, no benchmarks optimizados ni pruebas de metadatos dinámicos.

*Evaluación de rendimiento pendiente.* Estos resultados no prueban aceleración, recursos, tiempo físico ni suministro eficiente de metadatos dinámicos.

### A. Amenazas a la validez

El esqueleto compartido reduce el sesgo de optimización pero no demuestra optimalidad ni elimina la interacción software–arquitectura. La distribución sintética no representa necesariamente una red real. Con  $z=0$  puede omitirse la corrección separada, sin demostrar igualdad de rendimiento. Despachar entre inmediatos permitiría  $z$  dinámico sin otro encoding, pero sigue sin implementación ni costo medido. Los contadores requieren validación; ciclos RTL no demuestran timing físico.

## VI. CONCLUSIONES

*Pendiente de evaluación.* La pregunta y el alcance están delimitados, pero aún no se puede concluir que la fusión mejore el rendimiento. El siguiente paso es cerrar las condiciones de comparación y desarrollar kernels sobre la base común con MUL y las dos operaciones empacadas, antes de medir.

## REFERENCIAS

- [1] B. Jacob, S. Kligys, B. Chen, M. Zhu, M. Tang, A. Howard, H. Adam, and D. Kalenichenko, “Quantization and training of neural networks for efficient integer-arithmetic-only inference,” arXiv:1712.05877, 2017. [Online]. Available: <https://arxiv.org/abs/1712.05877>
- [2] A. Garofalo, G. Tagliavini, F. Conti, L. Benini, and D. Rossi, “XpulpNN: Enabling energy efficient and flexible inference of quantized neural network on RISC-V based IoT end nodes,” arXiv:2011.14325, 2020. [Online]. Available: <https://arxiv.org/abs/2011.14325>
- [3] G. Ottavi, A. Garofalo, G. Tagliavini, F. Conti, A. D. Mauro, L. Benini, and D. Rossi, “Dustin: A 16-cores parallel ultra-low-power cluster with 2b-to-32b fully flexible bit-precision and vector lockstep execution mode,” arXiv:2201.08656v2, 2023. [Online]. Available: <https://arxiv.org/abs/2201.08656v2>
- [4] RISC-V International, “The RISC-V Instruction Set Manual, Volume I: M extension, version 2.0,” ISA v20260120. <https://docs.riscv.org/reference/isa/v20260120/unpriv/m-st-ext.html>, 2026, accessed: 2026-09-08.